

LAB 12 — Performance Optimization & Deployment

Section A — Introduction

In this lab, students will apply the performance optimization techniques learned in **Module 12** to improve the **Taskly** application.

Students will analyze performance issues, reduce unnecessary widget rebuilds, optimize images and assets, analyze app size, and finally produce a **Release build** ready for distribution.

The lab consists of four exercises:

1. **Exercise 12.1 — Optimize List Rebuilds**
 2. **Exercise 12.2 — Image & Asset Optimization**
 3. **Exercise 12.3 — App Size Analysis**
 4. **Exercise 12.4 — Final Optimization & Deployment (Comprehensive)**
-

★ Section B — Lab Instructions

◇ Exercise 12.1 — Optimize List Rebuilds in Taskly

Objective

Reduce unnecessary widget rebuilds when adding/toggling/deleting tasks.

Tasks

1. Open the **Taskly** project used in Modules 11–12 (or your copied Module 12 version).
2. Review the current implementation: TaskListScreen + inline ListTile.
3. Apply optimizations:
 - Extract TaskTile into a separate widget
 - Use Selector<TaskProvider, List<Task>> instead of watching the entire screen
 - Add const to static widgets

- Assign keys properly using `ValueKey(task.id)`

Deliverables

- Before/After screenshots showing rebuild differences
 - Updated code files: `task_list_screen.dart` and `task_tile.dart`
-

◇ Exercise 12.2 — Image & Asset Optimization

Objective

Optimize images and assets to improve performance and reduce app size.

Tasks

1. Add a small test icon or image to Taskly (PNG recommended).
2. Perform optimizations:
 - Resize the image to an appropriate resolution (e.g., 128×128)
 - Pre-cache frequently used images
 - Remove unused assets from `pubspec.yaml`

Deliverables

- Before/After asset comparison
 - Code snippet showing `precacheImage()`
 - List of removed unused assets (if any)
-

◇ Exercise 12.3 — App Size Analysis

Objective

Analyze the app's size and identify sources of large file contributions.

Tasks

1. Run the size analysis command:
2. `flutter build apk --analyze-size`

3. Open and review the generated HTML size report.
4. Record:
 - Total APK size
 - Top 3 components taking the most space
 - Which assets or dependencies can be optimized or removed

Deliverables

- Screenshot of the size report
 - A written list of optimization suggestions
-

◇ **Exercise 12.4 — Final Optimization & Deployment (Comprehensive)**

Objective

Finalize optimization and generate a Release build ready for deployment.

Tasks

1. Run the app in **Profile mode**:
2. `flutter run --profile`

Observe FPS, jank, rebuild frequency, and responsiveness.

3. Apply final optimizations:
 - Remove unnecessary `print()` or debug logs
 - Add `const` where applicable
 - Remove unused assets/dependencies
 - Run `flutter clean`
4. Build a **Release APK**:
5. `flutter build apk --release`

Or build an **AppBundle** (recommended for Google Play):

`flutter build appbundle --release`

6. Install and test the Release APK on a physical device or emulator.

Deliverables

- Release APK or AAB file
 - Screenshot of Taskly running in Release mode
 - Completed performance checklist
 - Short summary: *“Why the app is now ready for deployment”*
-

★ Section C — Submission Format

Students must submit a .zip file containing:

1. Code Folder

lab12/

/taskly/

/lib/

/assets/

2. Report (PDF or DOCX)

- Results for exercises 12.1 → 12.4
- Screenshots
- Explanations & observations

3. Release Artifact

- app-release.apk or app-release.aab
-

★ Rubric (100%)

Component	Description	Weight
Exercise 12.1 – List Optimization	Widget extraction, Selector use, reduced rebuilds	25%
Exercise 12.2 – Image Optimization	Resizing, precaching, asset cleanup	15%
Exercise 12.3 – App Size Analysis	Size report findings & optimization proposals	20%
Exercise 12.4 – Final Optimization & Deployment	Profile testing, final cleanup, Release build	30%
Report Quality & Submission Format	Clear documentation, correct file structure	10%
Total: 100%		